

Tek Bantlı Turing Makinesi ile Binary Çarpma Hesaplayıcı

Bilgi	Açıklama
Ders	Özdevinirler Kuramı
Ödev	Final Ödev 1
Proje Adı	Tek Bantlı Turing Makinesi ile İkili Sayı Çarpma Makinesi
Öğrenci	Yasin Engin
Öğrenci No	23060510
Kullanılan Dil	Python
Teslim İçeriği	Kaynak kod, geçiş tablosu, durum diyagramı, test örnekleri, program ekran görüntüleri ve proje raporu
GitHub Linki	https://github.com/YasinEnginn/binary-multiplication-turing-machine
YouTube Video Linki	https://youtu.be/dtmzvePSPGo

Teslim edilecek PDF dosya adı 23060510_proje1_rapor.pdf biçiminde hazırlanmıştır.

Özet

Bu çalışmada, iki binary sayıyı çarpmak üzere tek bantlı bir Turing Makinesi simülatörü geliştirilmiştir. Program kullanıcıdan iki adet binary sayı alır, girdiyi $birinci * ikinci =$ biçiminde banda yerleştirir, * ve = sembolleri yardımıyla operandları ayrıştırır ve sonucu = sembolünden sonraki alana yazar.

Çarpma işlemi, bilgisayar mimarisinde de temel bir yaklaşım olan kaydır ve topla yöntemiyle modellenmiştir. Multiplier bitleri sağdan sola işlenir; bit 1 olduğunda multiplicand ilgili basamak kadar sola kaydırılarak mevcut sonuca eklenir, bit 0 olduğunda ise yalnızca kaydırma adımı ilerletilir. Program her adımda mevcut durumu, okunan sembolü, yazılan sembolü, kafa hareketini, sonraki durumu ve bant içeriğini raporlar.

1. Problem Tanımı

Ödevin amacı, binary sayı sistemi üzerinde çarpma işlemini Turing Makinesi seviyesinde modellemektir. Geliştirilen program şu temel gereksinimleri karşılar:

- kullanıcıdan iki binary sayı alınması
- girdilerin yalnızca 0 ve 1 sembollerinden oluştuğunun doğrulanması
- bant formatının $multiplicand * multiplier =$ olarak oluşturulması
- * sembolü ile operandların açık biçimde ayrıştırılması
- = sembolünden sonra sonuç alanının kullanılması
- çarpmanın shift & add yaklaşımıyla gerçekleştirilmesi
- her makine adımının izlenebilir biçimde çıktı olarak verilmesi
- sonucun hem binary hem de decimal karşılığıyla gösterilmesi

Ödevde verilen örnek:

```
Girdi bandı : 11*10=
Sonuç       : 110
Final bant  : 11*10=110
```

Bu sonuç doğrudur; çünkü $11_2 = 3$, $10_2 = 2$ ve $3 \times 2 = 6 = 110_2$.

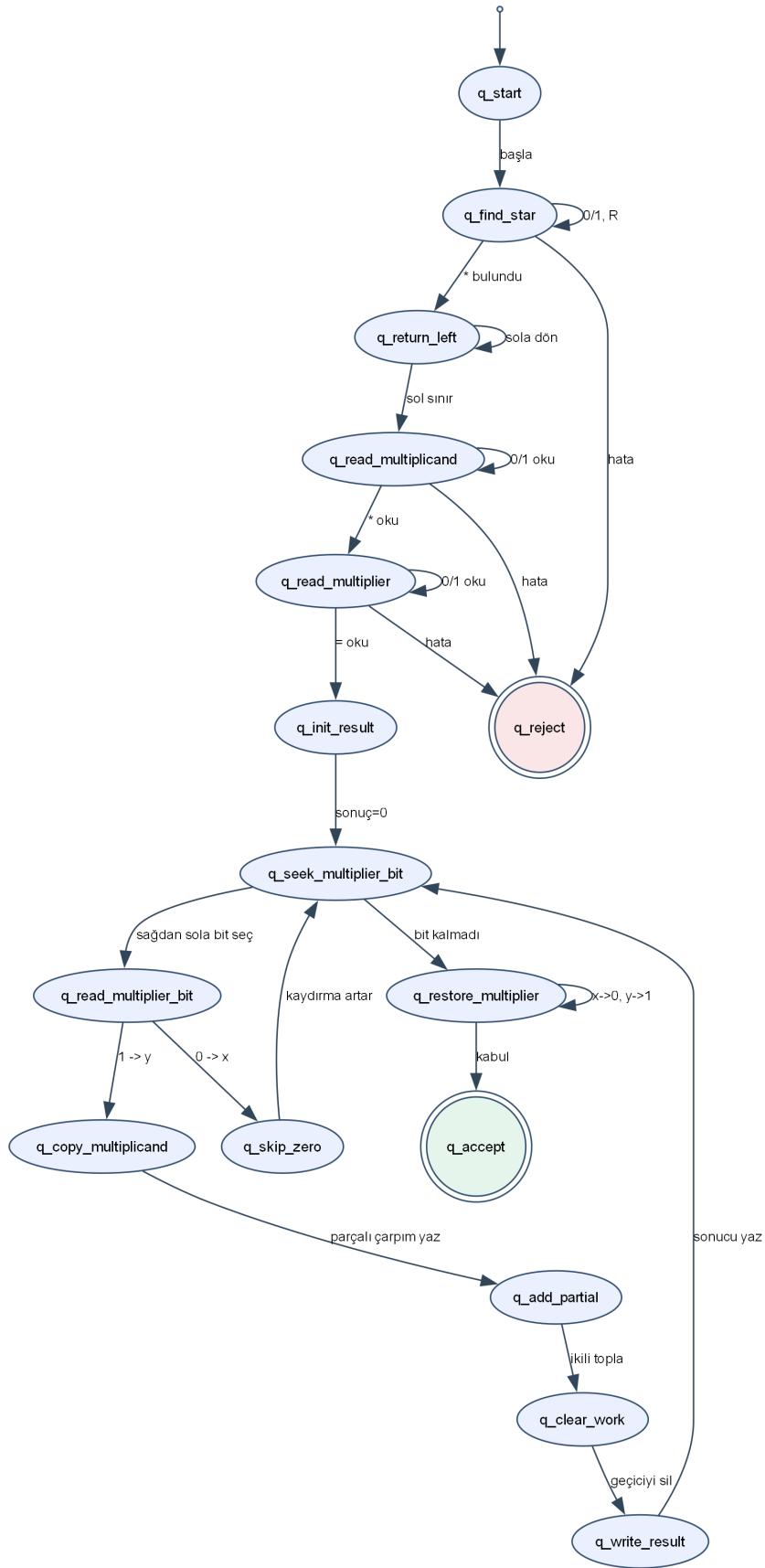
2. Turing Makinesi Modeli

Simülatör tek bant ve tek okuma-yazma kafası kullanır. Bant, sınırsız kabul edilen seyrek bir yapı olarak modellenmiştir; kullanılmayan hücreler `_` boş sembolüyle temsil edilir.

Bileşen	Tanım
Durum kümesi	<code>q_start</code> , <code>q_find_star</code> , <code>q_return_left</code> , <code>q_read_multiplicand</code> , <code>q_read_multiplier</code> , <code>q_init_result</code> , <code>q_seek_multiplier_bit</code> , <code>q_read_multiplier_bit</code> , <code>q_skip_zero</code> , <code>q_copy_multiplicand</code> , <code>q_add_partial</code> , <code>q_write_result</code> , <code>q_clear_work</code> , <code>q_restore_multiplier</code> , <code>q_accept</code> , <code>q_reject</code>
Giriş alfabesi	<code>{0, 1}</code>
Bant alfabesi	<code>{0, 1, *, =, #, x, y, _}</code>
Başlangıç durumu	<code>q_start</code>
Kabul durumu	<code>q_accept</code>
Red durumu	<code>q_reject</code>
Boş sembol	<code>_</code>
Kafa hareketleri	L sola, R sağa, S aynı hücrede kal

Geçiş tablosu `docs/gecis_tablosu.md` dosyasında, durum geçiş diyagramı ise `docs/durum_diyagrami.png` dosyasında sunulmuştur.

Şekil 1. Tek bantlı binary çarpma makinesinin ana durum geçişleri.



3. Binary Sayı Sistemi

Binary sayı sisteminde her basamak 2'nin kuvvetlerini temsil eder. En sağdaki basamak 2^0 , onun solundaki basamak 2^1 , sonraki basamak ise 2^2 değerine sahiptir.

Örnek:

$$101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$$

$$11_2 = 1 \cdot 2^1 + 1 \cdot 2^0 = 3$$

Bu nedenle $101_2 \times 11_2 = 1111_2$ olur; decimal karşılığı $5 \times 3 = 15$ tir.

4. Bant Tasarımı ve Operand Ayrıştırma

Operand ayrıştırma, bu ödevin en kritik gereksinimidir. Program operandları yalnızca kullanıcı girdisindeki sıraya göre varsaymakla kalmaz; bant üzerinde * sembolünü bizzat arar ve sınırları bu sembole göre belirler.

Başlangıç bant biçimi:

multiplicand*multiplier=

Örnek:

11*10=

```
^^ ^^
|  |
|  multiplier
multiplicand
```

Bu gösterimde:

- * sembolünün sol tarafı birinci sayı, yani multiplicand alanıdır.
- * sembolünün sağ tarafı ve = sembolünün sol tarafı ikinci sayı, yani multiplier alanıdır.
- = sembolünün sağ tarafı sonuç alanıdır.

Makine q_find_star durumunda * sembolünü bulur. Daha sonra q_return_left durumuyla sol başa döner, q_read_multiplicand durumunda birinci operandı, q_read_multiplier durumunda ise ikinci operandı okur. Multiplier bitleri işlenirken 0 bitleri geçici olarak x, 1 bitleri ise geçici olarak y sembolüyle işaretlenir. İşlem sonunda q_restore_multiplier durumu bu işaretleri tekrar özgün 0 ve 1 sembollerine dönüştürür. Böylece final bant operandları ve sonucu açık biçimde korur.

5. Çarpma Algoritması: Kaydır ve Topla

Makinenin kullandığı çarpma stratejisi shift & add yaklaşımıdır. Bu yöntem, binary çarpmanın her multiplier biti için bir parçalı çarpım üretmesi ilkesine dayanır.

Algoritma:

1. Sonuç alanı başlangıçta 0 yapılır.
2. Multiplier'ın en sağ bitinden başlanır.
3. Okunan bit 0 ise parçalı çarpım üretilmez; yalnızca shift değeri artırılır.
4. Okunan bit 1 ise multiplicand, mevcut shift değeri kadar sola kaydırılır.
5. Kaydırılmış değer mevcut sonuçla binary toplama kurallarına göre toplanır.
6. Güncellenmiş sonuç = sembolünden sonraki alana yeniden yazılır.
7. Tüm multiplier bitleri işlendiğinde makine kabul durumuna geçer.

Ödevde verilen örnek bu mantıkla şu şekilde ilerler:

```
  11
x 10
----
 00
```

110

110

6. Durumların Açıklaması

Durum	Görev
q_start	Makinenin başlangıç durumudur.
q_find_star	Bant üzerinde * sembolünü arar ve operand sınırını belirler.
q_return_left	Multiplicand okuması için bandın sol başına döner.
q_read_multiplicand	* sembolüne kadar birinci operandı okur.
q_read_multiplier	= sembolüne kadar ikinci operandı okur.
q_init_result	Sonuç alanını 0 ile başlatır.
q_seek_multiplier_bit	Multiplier bitlerini sağdan sola tarar.
q_read_multiplier_bit	Sıradaki multiplier bitini okur ve işaretler.
q_skip_zero	Bit 0 olduğunda toplama yapmadan sonraki adıma geçer.
q_copy_multiplicand	Bit 1 olduğunda kaydırılmış parçalı çarpımı geçici alana yazar.
q_add_partial	Geçici parçalı çarpımı mevcut sonuçla toplar.
q_write_result	Güncellenmiş sonucu sonuç alanına yazar.
q_clear_work	Geçici çalışma alanını temizler.
q_restore_multiplier	x ve y işaretlerini tekrar 0 ve 1 haline getirir.
q_accept	Hesaplamanın başarıyla tamamlandığını gösterir.
q_reject	Geçersiz giriş veya beklenmeyen sembol durumudur.

7. Geçiş Mantığı

Geçiş fonksiyonu, Turing Makinesi'nin her adımda hangi sembolü okuyacağını, hangi sembolü yazacağını, kafayı hangi yöne hareket ettireceğini ve hangi duruma geçeceğini tanımlar.

Örnek geçişler:

Mevcut durum	Okunan	Yazılan	Hareket	Sonraki durum	Anlam
q_find_star	0/1	aynı	R	q_find_star	* bulunana kadar sağa ilerlenir.
q_find_star	*	*	L	q_return_left	Operand ayracı bulunur.
q_read_multiplier_bit	0	x	S	q_skip_zero	0 biti işaretlenir, toplama yapılmaz.
q_read_multiplier_bit	1	y	S	q_copy_multiplicand	1 biti için parçalı çarpım üretilir.
q_restore_multiplier	y	1	R	q_restore_multiplier	İşaretlenmiş bit eski haline getirilir.

Tam tablo docs/gecis_tablosu.md dosyasında verilmiştir.

8. Program Çıktıları

Program, hesaplama sürecini adım adım raporlar. Aşağıdaki çıktı, ödevde verilen 11 x 10 örneğinin özetidir.

Girdi bandı: 11*10=

Operand ayrımı: 11 * 10

Adım adım simülasyon:

Adım 0001 | durum=q_start oku=1 yaz=1 hareket=S sonraki=q_find_star
| Baslangic durumundan * arama durumuna gecildi.
bant: 11*10=

Adım 0004 | durum=q_find_star oku=* yaz=* hareket=L
sonraki=q_return_left | * bulundu; operand ayrımı baslatildi.
bant: 11*10=

Adım 0018 | durum=q_read_multiplier_bit oku=0 yaz=x hareket=S sonraki=q_skip_zero
| Bit 0 okundu; 0 basamak kaydırma için toplama yok.
bant: 11*1x=0

Adım 0020 | durum=q_read_multiplier_bit oku=1 yaz=y hareket=S
sonraki=q_copy_multiplicand | Bit 1 okundu; 1 basamak kaydırılmış parçalı çarpım eklenecek.
bant: 11*yx=0

Sonuc:

Binary : 110

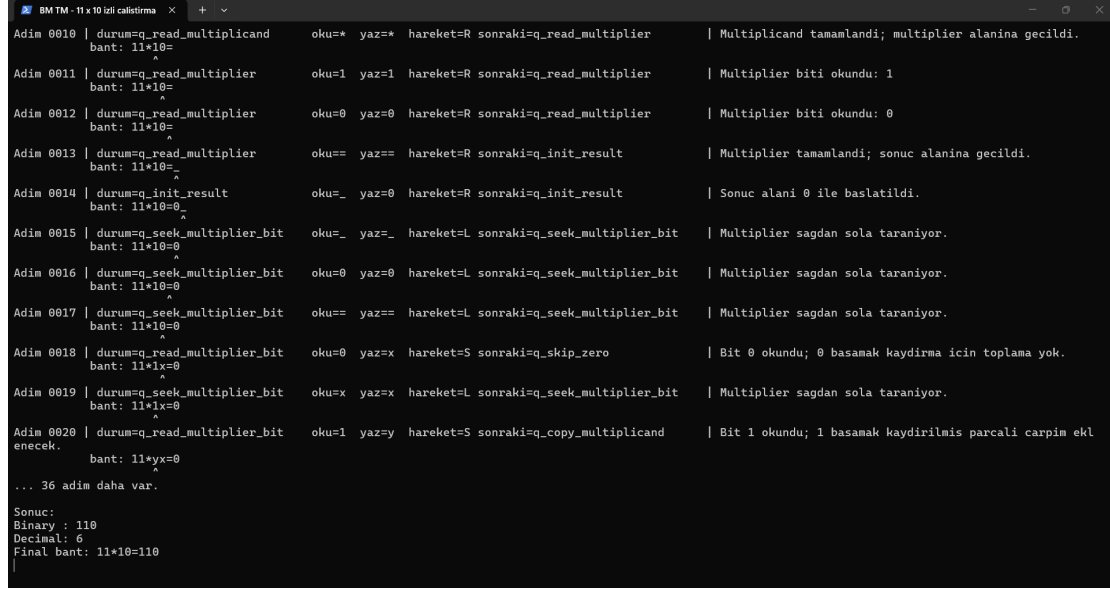
Decimal: 6

Final bant: 11*10=110

Daha fazla örnek çıktı docs/ornek_ciktilar.md dosyasında yer almaktadır.

9. Program Ekran Görüntüleri

Bu bölümde, programın ödev gereksinimlerini karşıladığını gösteren gerçek çalıştırma ekranları verilmiştir. Görseller doğrudan PowerShell penceresinde komutlar çalıştırıldıktan sonra alınmıştır.



```
BM TM - 11 x 10 bitli calistirma
Adim 0010 | durum=q_read_multiplicand      oku=*   yaz=*   hareket=R sonraki=q_read_multiplier | Multipland tamamlandi; multiplier alanina gecildi.
          bant: 11*10=
Adim 0011 | durum=q_read_multiplier        oku=1   yaz=1   hareket=R sonraki=q_read_multiplier | Multiplier biti okundu: 1
          bant: 11*10=
Adim 0012 | durum=q_read_multiplier        oku=0   yaz=0   hareket=R sonraki=q_read_multiplier | Multiplier biti okundu: 0
          bant: 11*10=
Adim 0013 | durum=q_read_multiplier        oku==   yaz==  hareket=R sonraki=q_init_result   | Multiplier tamamlandi; sonuc alanina gecildi.
          bant: 11*10=
Adim 0014 | durum=q_init_result            oku=_   yaz=0   hareket=R sonraki=q_init_result   | Sonucalani 0 ile baslatildi.
          bant: 11*10=0
Adim 0015 | durum=q_seek_multiplier_bit    oku=_   yaz=_   hareket=L sonraki=q_seek_multiplier_bit | Multiplier sagdan sola taraniyor.
          bant: 11*10=0
Adim 0016 | durum=q_seek_multiplier_bit    oku=0   yaz=0   hareket=L sonraki=q_seek_multiplier_bit | Multiplier sagdan sola taraniyor.
          bant: 11*10=0
Adim 0017 | durum=q_seek_multiplier_bit    oku==   yaz==  hareket=L sonraki=q_seek_multiplier_bit | Multiplier sagdan sola taraniyor.
          bant: 11*10=0
Adim 0018 | durum=q_read_multiplier_bit    oku=0   yaz=x   hareket=5 sonraki=q_skip_zero   | Bit 0 okundu; 0 basamak kaydirma icin toplama yok.
          bant: 11*1x=0
Adim 0019 | durum=q_seek_multiplier_bit    oku=x   yaz=x   hareket=L sonraki=q_seek_multiplier_bit | Multiplier sagdan sola taraniyor.
          bant: 11*1x=0
Adim 0020 | durum=q_read_multiplier_bit    oku=1   yaz=y   hareket=5 sonraki=q_copy_multiplicand | Bit 1 okundu; 1 basamak kaydirilmis parcali carpim eklenecek.
          bant: 11*yx=0
... 36 adim daha var.
Sonuc:
Binary : 110
Decimal: 6
Final bant: 11*10=110
```

Şekil 2. 11×10 girdisi için $*$ ayırıcının bulunması, multiplier bitlerinin işaretlenmesi ve sonucun banda yazılması.



```
BM TM - 101 x 11 kisa sonuc
Girdi bandi: 101*11=
Operand ayrimi: 101 * 11

Sonuc:
Binary : 1111
Decimal: 15
Final bant: 101*11=1111
```

Şekil 3. İz çıktısı kapalıyken programın binary, decimal ve final bant sonucunu özetlemesi.



```
BM TM - 0 x 1011
Girdi bandi: 0*1011=
Operand ayrimi: 0 * 1011

Sonuc:
Binary : 0
Decimal: 0
Final bant: 0*1011=0
```

Şekil 4. 0×1011 örneğinde sonucun 0 olarak üretildiğini gösteren çalıştırma ekranı.



```
BM TM - 1 x 11101
Girdi bandi: 1*11101=
Operand ayrimi: 1 * 11101

Sonuc:
Binary : 11101
Decimal: 29
Final bant: 1*11101=11101
```

Şekil 5. 1×11101 örneğinde ikinci operandın sonuç olarak korunduğunu gösteren çalıştırma ekranı.

```
BM TM - 1001 x 101
Girdi bandı: 1001*101=
Operand ayrımı: 1001 * 101

Sonuc:
Binary : 101101
Decimal: 45
Final bant: 1001*101=101101
```

Şekil 6. 1001 x 101 çok bitli çarpma örneğinin binary ve decimal sonucunu gösteren çalıştırma ekranı.

```
BM TM - 111 x 111
Girdi bandı: 111*111=
Operand ayrımı: 111 * 111

Sonuc:
Binary : 110001
Decimal: 49
Final bant: 111*111=110001
```

Şekil 7. 111 x 111 örneğinde final bant formatının korunduğunu gösteren çalıştırma ekranı.

```
BM TM - geçersiz girdi
Hata: Birinci sayı yalnızca 0 ve 1 karakterlerinden oluşmalıdır.
```

Şekil 8. Binary olmayan karakter içeren girişin program tarafından reddedildiğini gösteren hata ekranı.

```
BM TM - geçersiz ikinci girdi
Hata: İkinci sayı yalnızca 0 ve 1 karakterlerinden oluşmalıdır.
```

Şekil 9. İkinci operandda binary olmayan karakter bulunduğunda programın girişi reddetmesi.

```
BM TM - 101 x 11 izli çalıştırma
Adım 0008 | durum=q_return_left      oku=1 yaz=1 hareket=L sonraki=q_return_left | Sol sınır aranarak birinci operandın başına donuluyor.
           | bant: 101*11=
           | ^
Adım 0009 | durum=q_return_left      oku=_ yaz=_ hareket=R sonraki=q_read_multiplicand | Sol sınır bulundu; multiplicand okunacak.
           | bant: 101*11=
           | ^
Adım 0010 | durum=q_read_multiplicand  oku=1 yaz=1 hareket=R sonraki=q_read_multiplicand | Multiplicand biti okundu: 1
           | bant: 101*11=
           | ^
Adım 0011 | durum=q_read_multiplicand  oku=0 yaz=0 hareket=R sonraki=q_read_multiplicand | Multiplicand biti okundu: 0
           | bant: 101*11=
           | ^
Adım 0012 | durum=q_read_multiplicand  oku=1 yaz=1 hareket=R sonraki=q_read_multiplicand | Multiplicand biti okundu: 1
           | bant: 101*11=
           | ^
Adım 0013 | durum=q_read_multiplicand  oku=* yaz=* hareket=R sonraki=q_read_multiplier | Multiplicand tamamlandı; multiplier alanına geçildi.
           | bant: 101*11=
           | ^
Adım 0014 | durum=q_read_multiplier  oku=1 yaz=1 hareket=R sonraki=q_read_multiplier | Multiplier biti okundu: 1
           | bant: 101*11=
           | ^
Adım 0015 | durum=q_read_multiplier  oku=1 yaz=1 hareket=R sonraki=q_read_multiplier | Multiplier biti okundu: 1
           | bant: 101*11=
           | ^
Adım 0016 | durum=q_read_multiplier  oku== yaz== hareket=R sonraki=q_init_result | Multiplier tamamlandı; sonuç alanına geçildi.
           | bant: 101*11=_
           | ^
Adım 0017 | durum=q_init_result      oku=_ yaz=0 hareket=R sonraki=q_init_result | Sonuç alanı 0 ile başlatıldı.
           | bant: 101*11=0_
           | ^
Adım 0018 | durum=q_seek_multiplier_bit  oku=_ yaz=_ hareket=L sonraki=q_seek_multiplier_bit | Multiplier sağdan sola taranıyor.
           | bant: 101*11=0_
           | ^
... 83 adım daha var.

Sonuc:
Binary : 1111
Decimal: 15
Final bant: 101*11=1111
```

Şekil 10. 101 x 11 örneğinin izli çalıştırmasında operand okuma ve sonuç yazma adımlarının görünmesi.

10. Test Örnekleri

Program farklı girişler üzerinde test edilmiştir. Testler hem doğru sonucu hem de operand ayraçlarının final bantta korunup korunmadığını denetler.

No	Birinci sayı	İkinci sayı	Beklenen bi- nary sonuç	Decimal sonuç
1	11	10	110	6
2	0	0	0	0
3	0	1011	0	0
4	1	11101	11101	29
5	101	11	1111	15
6	1001	101	101101	45
7	111	111	110001	49

Test komutu:

```
python -m unittest -v
```

Son çalıştırmada 9 test başarıyla tamamlanmıştır.

```

test_assignment_example (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_assignment_example) ... ok
test_binary_addition_helper (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_binary_addition_helper) ... ok
test_invalid_input_is_rejected (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_invalid_input_is_rejected) ... ok
test_large_sample (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_large_sample) ... ok
test_multi_bit_operands (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_multi_bit_operands) ... ok
test_one_times_number (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_one_times_number) ... ok
test_operand_delimiters_are_preserved (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_operand_delimiters_are_preserved) ... ok
test_zero_times_nonzero (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_zero_times_nonzero) ... ok
test_zero_times_zero (tests.test_binary_multiplier_tm.BinaryMultiplicationTMTTests.test_zero_times_zero) ... ok

-----
Ran 9 tests in 0.005s

OK

```

Şekil 11. Birim testlerin tamamının başarıyla geçtiğini gösteren ekran çıktısı.

11. Sonuç ve Değerlendirme

Bu proje, binary çarpma işlemini tek bantlı Turing Makinesi modeliyle izlenebilir ve doğrulanabilir biçimde gerçekleştirmektedir. Çözümün merkezinde * ve = sembolleriyle yapılan açık operand ayrıştırması bulunmaktadır. Bu ayrıştırma sayesinde makine, * sembolünün solundaki değeri multiplicand, * ile = arasındaki değeri multiplier, = sembolünün sağını ise sonuç alanı olarak kullanır.

Çarpma işlemi kaydır ve topla yaklaşımıyla modellenmiş, her multiplier biti için gerekli işlem açık durum geçişleriyle temsil edilmiştir. Programın adım adım çıktı üretmesi, Turing Makinesi'nin soyut hesaplama sürecini gözlemlenebilir hale getirmektedir. Yapılan testler sonucunda programın farklı binary girişler için doğru sonuç ürettiği ve final bant formatını koruduğu doğrulanmıştır.